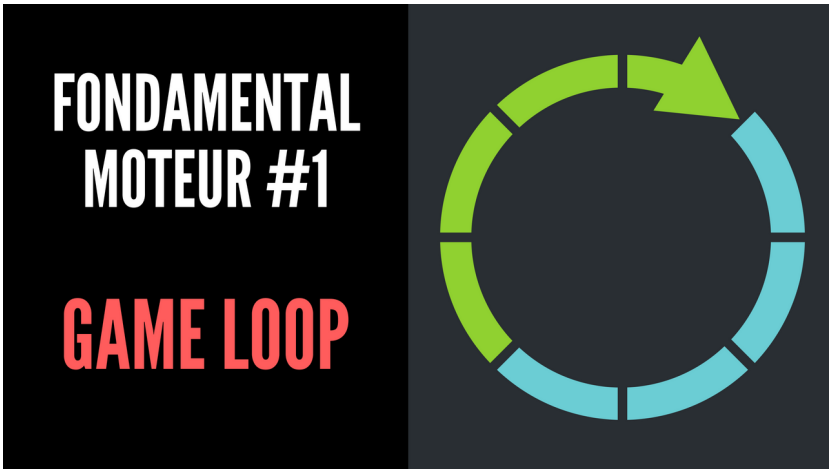
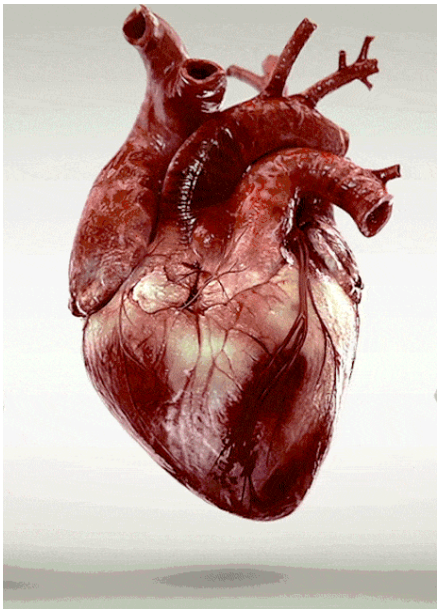


La Game Loop est le coeur de votre jeu vidéo

La Game loop (littéralement "la boucle de jeu"), c'est le coeur qui donne la vie de votre jeu vidéo.

Imaginez le coeur humain :



Le coeur humain "tourne en boucle", comme le fait votre jeu vidéo.

A chaque battement de coeur, votre corps réalise une série d'opérations vitales, comme le fait votre jeu vidéo.

Sans Game loop, votre jeu s'arrêterait de fonctionner, c'est comme si le coeur ne battait qu'une seule fois !



Quand j'avais 13 ans et que j'ai lancé la première cassette de programme dans mon Amstrad CPC, j'ai tout de suite cherché à comprendre comment le jeu pouvait continuer à fonctionner alors que la cassette ne tournait plus ! (oui, en 1984, je ne connaissais que les magnétoscopes, et niveau technologie c'était l'âge de pierre...).

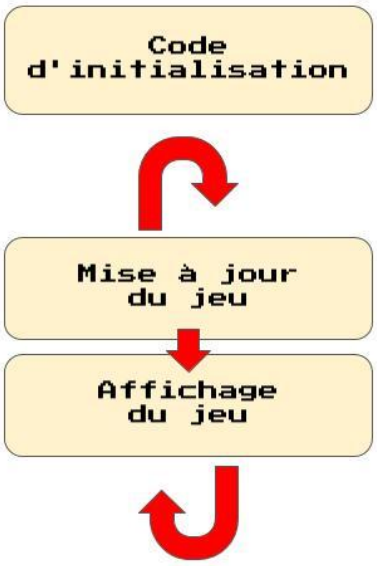
J'avais cru au départ, que comme pour un magnétoscope, l'image était enregistrée sur la cassette… Mais je ne comprenais pas comment j'arrivais à faire changer de direction à mon personnage car si l'image était pré-enregistrée, il aurait été impossible de faire cela !

En réalité, bien entendu, la cassette contenait un programme (enregistré sous forme de son, comme le bruit des Modems des années 90) qui était simplement chargé en mémoire puis exécuté.

Mine de rien, cette réflexion m'a rapidement permis de comprendre le secret du fonctionnement d'un jeu vidéo : la Game Loop !

Pour qu'un jeu, ou même n'importe quel programme, puisse fonctionner sans s'interrompre, il faut qu'il "boucle". Et l'image du jeu n'est pas pré-enregistrée, c'est le programmeur qui dessine le jeu à chaque boucle en fonction de ce qui se passe dans le jeu !

Voici comment se passe l'exécution de votre jeu :



Tout d'abord le code d'initialisation de votre jeu est exécuté. Il peut s'agir d'un code tout simplement positionné en dehors de toute fonction, ou dans une fonction prévue par le Framework à cet usage.

Puis à chaque boucle, le programmeur est en effet chargé de :

- Modifier l'état du jeu (par exemple déplacer de 1 pixel un personnage, changer son animation, tester une collision)

- Re-afficher la totalité du jeu en fonction de ce nouvel état !

Oui, vous lisez bien. On efface tout est on recommence à chaque boucle !



Sur mon Amstrad, ce n'était pas si simple. L'affichage était très lent. Donc le programmeur devait tricher et ne re-dessiner qu'une partie du jeu. Uniquement ce qui avait changé. Mais cela posait beaucoup de soucis, notamment les superpositions (quand le personnage

avance, il passe devant un décor, qu'il faut redessiner à l'endroit où le personnage est passé !). C'était le casse tête des programmeurs de l'époque.

Aujourd'hui, la puissance de cartes graphiques permet d'oublier tout ça. Redessiner tout l'écran se faire un en battement de cil !

Quand on utilise un framework, c'est au programmeur de s'occuper de tout cela. C'est idéal, non seulement pour réellement apprendre à programmer un jeu vidéo, mais aussi en terme de liberté de logique de programmation. C'est finalement assez naturel de travailler ainsi.

Quand on utilise un moteur, cette boucle est cachée. Cela pose des problèmes de performance car pour exécuter quelque chose à chaque boucle, on doit s'y "insérer" et ça ne se fait pas de manière naturelle. La plupart du temps on doit oublier notre game loop bien aimée et travailler avec les moyens du bord. Snif.

Mais sinon, ça marche comment en vrai ?

Rentrons un peu plus dans le détail. Je vous parlais de "Modifier l'état" et de "Re-afficher"...

On parlera plus précisément de "Mise à jour (update)" et d'affichage (draw).

Voici un exemple simplifié et en pseudo code :

```
Initialisation :
    Position X du personnage = 10
    Position Y du personnage = 100
Fin de Initialisation

Mise à jour :
    Si le joueur appuie sur la touche "droite" alors
        Ajoute 1 à la position X du personnage
    Fin de Si
Fin de Mise à jour

Affichage :
   Efface l'écran
   Affiche le décor
   Affiche le personnage à sa position X et Y
Fin de Mise à jour
```

Dans certains Frameworks, la distinction entre mise à jour et affichage n'est pas faire. Une seule fonction est appelée et elle doit s'occuper de tout.

Je vous conseille de toujours séparer Mise à jour et Affichage, quitte à vous créer 2 fonctions pour ça, et à les appeler l'une après l'autre dans la Game Loop.

Dans cet exemple, le framework va exécuter le code d'initialisation, puis à l'infini un enchaînement de Mise à jour et Affichage.

Si le joueur presse la touche "flèche droite", la position du personnage va changer (il avance vers la droite de 1 pixel). Et ce changement de position sera visible au moment de l'affichage qui suit !

Comment se termine le programme me direz-vous ?

C'est dans la plupart des cas au programmeur de prévoir un condition de sortie (exemple, touche ESC ou bien le joueur clique dans un menu pour quitter). Dans ce cas, il suffit de "casser" la boucle pour en sortir. Ceci sera abordé dans la partie pratique.

Vous savez tout !

Vous avez maintenant une bonne vision théorique du principe… Alors on passe à l'action avec de la pratique afin de comprendre comment ceci est mis en oeuvre dans votre moteur 2D.

Des cours spécifiques Love2D et Monogame sont disponibles dans les parcours correspondants. A vos claviers !